

Reviewer Pack — Symmetric Power Decomposition Complexity Gap for 3-SAT

Entry 9f467f497a7b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 03:39:36 UTC

Symmetric Power Decomposition Complexity Gap for 3-SAT

Entry ID: 9f467f497a7b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 03:39:36 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): 3-SAT Instances

Statement:

For a 3-SAT instance with n variables, the number of irreducible components in the symmetric power decomposition of its clause-variable incidence matrix is $\Omega(2^n)$ larger than for the corresponding determinant's decomposition.

Rationale (proposer's reasoning):

Schur-Weyl duality decomposes tensor powers into irreducible representations; permanent's symmetric nature yields exponentially more components than determinant's alternating structure, exposing algebraic barriers to polynomial identity testing.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 384f9def6b1f4a90

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    def is_prime(n):
        if n <= 1:
            return False
        for i in range(2, int(math.sqrt(n)) + 1):
            if n % i == 0:
                return False
        return True

    def generate_3sat_instance(n: int) -> list:
        clauses = []
        for _ in range(n):
            clause = [random.choice(range(-n, n+1)) for _ in range(3)]
            clauses.append(clause)
        return clauses

    def incidence_matrix(clauses: list, n: int) -> list:
        matrix = [[0] * n for _ in range(len(clauses))]
        for i, clause in enumerate(clauses):
            for var in clause:
                if var > 0:
                    matrix[i][var-1] = 1
                else:
                    matrix[i][-var-1] = -1
        return matrix

    def hook_length_form(n: int) -> int:
        hook_lengths = [n + i - j for i in range(1, n+1) for j in range(i)]
        product = math.prod(hook_lengths)
        factorial_n = math.factorial(n)
        return product // (factorial_n ** 2)

    def irreducible_components(n: int, k: int) -> int:
        if k == 0:
            return 1
        return sum([hook_length_form(i) * hook_length_form(j) for i in range(1, n+1) for j in range(1, n+1)])

    def permanent(matrix: list) -> int:
        if len(matrix) == 0:
            return 1
        det = 0
        for i in range(len(matrix)):
```

```

        submatrix = [row[:i] + row[i+1:] for row in matrix[1:]]
        sign = (-1) ** i
        det += sign * matrix[0][i] * permanent(submatrix)
    return det

def determinant(matrix: list) -> int:
    if len(matrix) == 0:
        return 1
    det = 0
    for i in range(len(matrix)):
        submatrix = [row[:i] + row[i+1:] for row in matrix[1:]]
        sign = (-1) ** i
        det += sign * matrix[0][i] * determinant(submatrix)
    return det

n = 20
k = n // 2
random.seed(seed)
instance = generate_3sat_instance(n)
matrix = incidence_matrix(instance, n)

perm_components = irreducible_components(n, k)
det_components = irreducible_components(n, k)

metric_name = "Symmetric Power Decomposition Complexity Gap"
metric_value = perm_components - det_components
instances_tested = 1
conjecture_holds = metric_value >= 2 ** n
counterexample = "" if conjecture_holds else f"Permanent components: {perm_components}, Determinant components: {det_components}"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]]
    if not seeds:
        primes = []
        num_primes = 30
        candidate = 2
        while len(primes) < num_primes:
            if is_prime(candidate):
                primes.append(candidate)
            candidate += 1
        seeds = primes

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    counterexample_desc = results[first_failing_seed]["counterexample"]
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample_desc}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

79426043297591592576425399311708450748681513839438964967856509413642281551708967083891786846523124
 TRIAL: {'metric_name': 'Symmetric Power Decomposition Complexity Gap', 'metric_value': 0, 'instances': 1}

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing valid data, rendering the metric value unreliable. | next:
 Re-run test with increased time limits and verify code stability

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	88126
2	preregistration	ollama_remote	qwen3:8b	0	0	24145
3	novelty	ollama_remote	qwen3:8b	0	0	20807
4	novelty	ollama_remote	qwen3:8b	0	0	18499
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11774
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11962
7	judge	ollama_remote	qwen3:8b	0	0	19178

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 194492 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/9f467f497a7b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9f467f497a7b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9f467f497a7b.tar.gz` (if generated)